

PMSCS 622P

Programming Languages

Lecture 09: Searching and Sorting Algorithms

Mr. Md. Masum Bhuiyan

Department of Computer Science and Engineering
Jahangirnagar University

Learning Objectives

- Today, we focus purely on **Algorithmic Logic**.
- We have removed all C++, Java, and Python code from this deck.
- Instead, we will trace how the computer "thinks" using step-by-step visual diagrams.

What is an Algorithm?

Definition

An algorithm is a step-by-step set of operations to be performed.

Algorithms are language-agnostic. The logic of finding a number is the exact same whether you speak English, Spanish, Python, or Java.

Searching vs. Sorting

Searching

- *Goal:* Find a specific item.
- *Input:* A list and a "target".
- *Output:* The location (index).

Sorting

- *Goal:* Order the items.
- *Input:* A scrambled list.
- *Output:* An ordered list.

Part 1: Linear Search

Linear Search: Concept

How it works:

- Start at the very beginning of the list (Index 0).
- Look at the item. Is it the target?
- If yes, STOP.
- If no, move right by exactly one position and repeat.

Linear Search: The Toy Problem

Target to Find: **7**



Linear Search: Step 1 (Initialize)

Target: 7

Current



Is $4 == 7$?

Linear Search: Step 2 (Evaluate)

Target: 7

Current

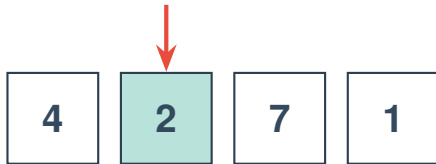


NO. Move to the next index.

Linear Search: Step 3 (Move)

Target: 7

Current

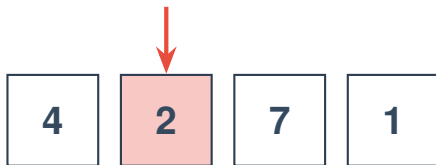


Is $2 == 7$?

Linear Search: Step 4 (Evaluate)

Target: 7

Current

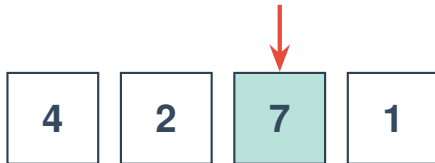


NO. Move to the next index.

Linear Search: Step 5 (Move)

Target: 7

Current

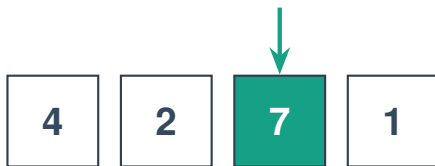


Is $7 == 7$?

Linear Search: Step 6 (Evaluate & Stop)

Target: 7

FOUND!



YES. Return Index 2. Algorithm terminates.

Part 2: Binary Search

Binary Search Concept

How it works:

- Look at the middle item.
- If it's the target, STOP.
- If the target is smaller, discard the right half.
- If the target is larger, discard the left half.

CRITICAL WARNING

This ONLY works if the array is already sorted!

Binary Search: The Toy Problem

Target to Find: **7**



Binary Search: Set Boundaries

Target: 7

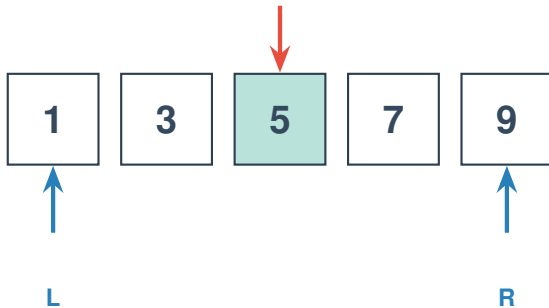


We establish a Left boundary (index 0) and Right boundary (index 4).

Binary Search: Calculate Middle

Target: 7

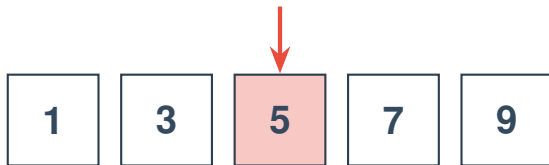
$$\text{Mid} = (0+4)/2 = 2$$



Binary Search: Evaluate Middle

Target: 7

Mid = 2



Is $5 == 7$? **No.**

Is $5 < 7$? **Yes!**

Binary Search: Discard Half

Target: 7

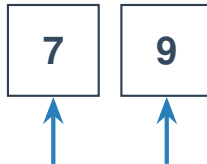


Discarded

Because the array is sorted, if 5 is too small, everything to the left of 5 is also too small!

Binary Search: Update Boundaries

Target: 7

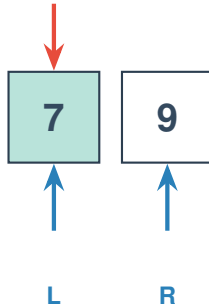


Left (L) = Mid - 1
Right (R) = 4

Binary Search: Calculate Middle (Again)

Target: 7

$$\text{Mid} = (3+4)/2 = 3.5 \rightarrow 3$$



Binary Search: Evaluate Middle (Again)

Target: 7

FOUND!



Is $7 == 7$? **YES.** Return index **3.**

Part 3: Bubble Sort

Bubble Sort Concept

How it works:

- Compare adjacent elements (side-by-side).
- If the left one is bigger, swap them!
- Move one position right and repeat.
- By the end of one pass, the largest number "bubbles" to the far right.
- Repeat passes until no swaps are needed.

Bubble Sort: The Toy Problem

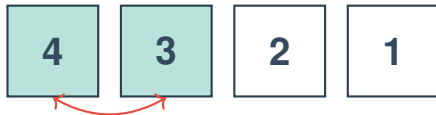
Goal: Sort in Ascending Order (Small to Large)



This is the worst-case scenario: perfectly reversed! Let's trace it.

Bubble Sort: Pass 1, Compare 0 & 1

Pass 1



Is $4 > 3$? **YES**. Swap them.

Bubble Sort: Pass 1, Swap Result

Pass 1



Swapped!

Bubble Sort: Pass 1, Compare 1 & 2

Pass 1



Is $4 > 2$? **YES.** Swap them.

Bubble Sort: Pass 1, Swap Result

Pass 1



Swapped!

Bubble Sort: Pass 1, Compare 2 & 3

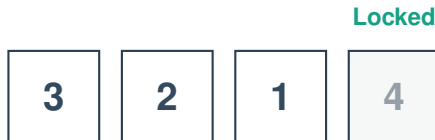
Pass 1



Is $4 > 1$? **YES.** Swap them.

Bubble Sort: End of Pass 1

Pass 1 Complete



The largest number (4) has "bubbled" to its correct final position. We don't need to check it again!

Bubble Sort: Pass 2, Compare 0 & 1

Pass 2



Start from the beginning. Is $3 > 2$? **YES**. Swap.

Bubble Sort: Pass 2, Swap Result

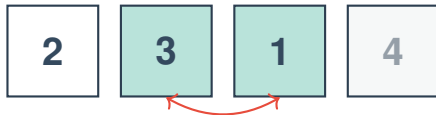
Pass 2



Swapped!

Bubble Sort: Pass 2, Compare 1 & 2

Pass 2



Is $3 > 1$? **YES**. Swap.

Bubble Sort: End of Pass 2

Pass 2 Complete



The next largest number (3) has bubbled to its spot.

Bubble Sort: Pass 3, Compare 0 & 1

Pass 3



Start from the beginning. Is $2 > 1$? **YES**. Swap.

Bubble Sort: End of Pass 3

Pass 3 Complete

Locked



The number 2 is locked.

Bubble Sort: Final State

Fully Sorted!

Locked



If the last 3 items in a 4-item list are sorted, the first item **MUST** be in the correct place by default. The algorithm is finished!

Summary of Visual Logic

- **Linear Search:** A simple pointer moving left to right. Checks every item. Reliable but slow.
- **Binary Search:** Uses Left, Right, and Mid pointers. Divides the search space in half each time. Extremely fast, but requires sorted data.
- **Bubble Sort:** Compares adjacent items and swaps them if they are out of order. Requires multiple "passes" to lock elements into place.